Member-only story

How to fine-tune an LLM

How I took the entire Singapore legislation and used it to fine-tune the Llama-3.1–8B-Instruct LLM

31 min read · Jul 13, 2025

Sau Sheong Follow

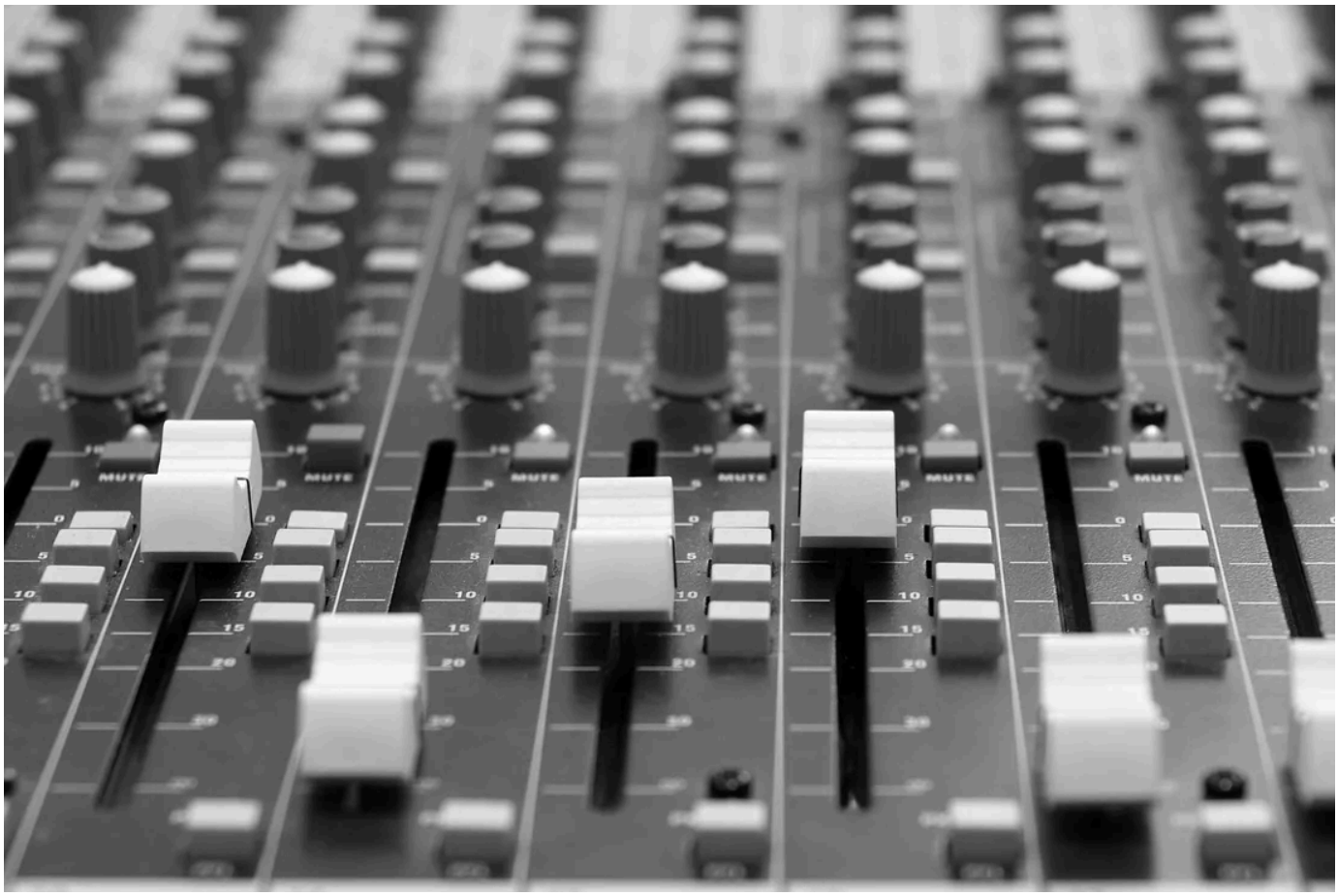

Photo by Scott Platt: https://www.pexels.com/photo/grayscale-photo-photo-of-an-audio-mixer-6134552/

I had always thought LLMs are the perfect tools for the legal industry. Law is mostly about documents — reading, writing, and interpreting complex legal texts like contracts, statutes, and court decisions and LLMs are designed to understand and generate text.

LLMs can quickly summarize documents, find relevant information, and answer legal questions. This makes them ideal for handling the heavy reading and writing that lawyers deal with every day. They can also help save time by automating repetitive tasks like reviewing contracts, doing legal research, or sorting through large volumes of documents during a lawsuit. Instead of spending hours on these jobs, legal professionals can use LLMs to speed up the process and focus on more important work.

LLMs can also be trained on specific legal data — like a country's laws or court cases — so they become even more accurate and useful for that legal system. And this is precisely what I tried to do.

In this article I will show you how to fine-tune the Llama-3.1–8B-Instruct LLM with the entire Singapore legislation and infer responses using it. Essentially there there 3 steps to doing this:

1. Creating the fine-tune dataset

- Download all the statutes and subsidiary legislation from the Singapore Statutes Online. They are available in PDF files
- Extract the content from each file and save them into a bunch of corresponding JSON files each containing one piece of legislation
- Convert the JSON files into an Alpaca format JSONL file, which is compatible with the Huggingface dataset package. This will be our fine-tune dataset

2. Fine-tune the LLM

- Use the fine-tune dataset and fine-tune Meta's Llama-3.1–8B-Instruct model with Parameter Efficient Fine-Tuning (PEFT) with LoRA adapters, using the Unsloth fine-tuning tool
- Merge the LoRA adapter with the base model to create the fine-tuned model

3. Infer with fine-tuned model

Finally, we can use fine-tuned model using an inference script.

Let's get started!

Creating the fine-tune dataset

Creating the dataset for fine-tuning is the first step. This is not a trivial task by any means — it requires careful data extraction, cleaning, formatting, and thinking through the example generation to ensure the model learns effectively. Poorly structured or inconsistent data can lead to suboptimal results, even if the model architecture and training configuration are sound.

The quality and diversity of the dataset directly influence the model's ability to generalize and follow instructions accurately. In domains like law, where precision and context are critical, it's especially important to generate examples that cover a range of question types and provide clear, reliable answers. This work determines whether the fine-tuned model will be genuinely useful.

Creating such datasets involves balancing automation with oversight — automated scripts can efficiently generate large volumes of data, but human review and domain knowledge are often necessary to validate correctness and relevance. Investing time and care at this stage pays dividends later by producing a model that is not only accurate, but also trustworthy and aligned with the needs of its intended users.

Having said all that, creating such datasets is the toughest and most tedious part of fine-tuning an LLM, generally taking between 70–90% of the total effort and time spent. Despite what I just said, I won't show any code I used to create the dataset here because it would take up quite a bit of space and it's really not the focus of this article. Generally speaking I used whichever language and library that works well for me — Python to download the PDFs and Go to process them.

I also used LLMs extensively to process the files as it's easier with a large number of files. For this, I used the `gemma3-4B` model running Ollama on my laptop.

Download the legislation

You can find the entire Singapore legislation online at the Singapore Statutes Online, which is a free service provided to the public by the Legislation Division of the Attorney-General's Chambers of Singapore.

Former Supreme Court, Singapore by Kok Leng Yeo

While the content is free, they are in PDFs scattered all over the site. Also each statute can have subsidiary legislation so it's pretty much everywhere. But at least they are nicely packaged in reasonably formatted PDFs.

Downloading each file by hand is probably the quickest way to stop the project. Fortunately we can always automate the task, and for this I simply inspected the HTML to figure out what to download and created a script to download the PDFs.

Like many scripts that deal with web scraping o r parsing HTML, I used the `BeautifulSoup` library to sieve through the ocean of tags and grab the correct URLs. The rest are just standard Python libraries. If all works well, you should end up downloading 3,581 PDF files, each containing either a statute or subsidiary legislation.

Extract content

Next, we want to extract the content and convert it into a JSON format. This is actually an intermediate step, ultimately we just want the fine-tune dataset in JSONL but to get there, it's easier to have an intermediate JSON file that we can also eye-ball for issues.

However, converting this to JSON itself is not a trivial task. This is because the files are not uniformly formatted the same way, though many are similarly enough. Also, while some of the files are relatively small, some can be really big. For example, the Income Tax Act 1947 has 1,260 pages, The Securities and Futures Act 2001 has 1,086 pages and so on.

Income Tax Act 1947

ARRANGEMENT OF SECTIONS

PART 1

PRELIMINARY

The first Arrangement of Sections page from the Income Tax Act 1947 document

Most of the acts are structured into multiple parts, which in turn are split into multiple sections. Each section can also be split into subsection but in order to keep things simple, I just leave it at the section level. In other words, each JSON file should look somewhat like this (of course with a lot more content):

```
{
  "title": "INCOME TAX ACT 1947",
  "parts": [
    {
      "title": "PRELIMINARY",
      "sections": [
        {
          "title": "1. Short title",
          "text": "This Act is the Income Tax Act 1947."
        }
      ]
    }
  ]
}
```

This sounds easy but it's not as straightforward. I ended up having to convert the PDF files first to text, then split it by parts, then use an LLM to process each part into sections and finally putting those sections back together again. This can be difficult for some files because not all files follow the same structure so I even resorted to manually splitting up certain files.

Convert into JSONL

The Hugging Face dataset format is a standardized way to represent and work with datasets using the `datasets` library, especially for training and fine-tuning language models. It treats data as a collection of examples, where each example is a dictionary with named fields like "instruction", "input", and "output". This format is efficient, easy to manipulate, and integrates well with other Hugging Face tools such as transformers, tokenizers, and Trainer.

For instruction-following tasks, the dataset typically includes three fields:

1. A instruction (which tells the model what to do)
2. An input (the content or context for the task), and
3. An output (the expected response).

This structure supports a wide range of tasks like summarization, translation, and question answering. I'm using the Alpaca format, which follows this structure, because it's simple, effective, and widely adopted for instruction tuning. It's also well-supported by open-source training scripts and aligns closely with the prompt-response style used by modern LLMs.

This format became popular with the release of the Alpaca model by Stanford, which demonstrated that strong instruction-following behavior could be achieved using a small amount of supervised data and open-source tools. The Alpaca format closely mimics a conversational prompt-response structure, making it ideal for training helpful, general-purpose assistants. It's widely used because it's straightforward, compatible with Hugging Face tools, and adaptable to many kinds of tasks.

The Alpaca format is typically stored and shared using the JSONL (JSON Lines) file format, where each line in the file is a standalone JSON object representing one example in the dataset. This makes it easy to process large datasets line-by-line without loading the entire file into memory.

This is an example of a JSONL file.

```
{"instruction": "Translate to Spanish", "input": "Hello", "output": "Hola"}
{"instruction": "Summarize the paragraph", "input": "Large language models are...", "output": "LLMs are powerful AI tools."
```

The conversion from the JSON into the Huggingface dataset format requires more LLM use. The strategy is to take each of the JSON file previously created and generate a bunch of instructions, with the given input and the expected output. Again, this seems straightforward but it can be quite time consuming. Generating the instructions are done using an LLM (gemma3-4B) and it works surprisingly well.

I ended up with about 200,000 sets of instructions for the 3,581 PDF files. Some files have just 1–2 questions, others went up to as many as 2,000 instructions. As you can imagine, the processing took quite a bit of time.

Once all examples are generated, I compile them into a structured dataset and splits the data into training (90%) and validation (10%) sets. It saves the output in several formats: a Huggingface-compatible dataset for machine learning pipelines, JSON files for easier manual inspection, and summary statistics showing the volume and distribution of generated samples.

The fine-tune dataset is now ready!

Fine-tune Llama-3.1–8B-Instruct

This is where the action starts. It might sound daunting but actually it's quite straightforward. However it's a bit like chess — it's easy to learn but difficult to master. In this case, the steps to fine-tuning are quite straightforward but it's hard to get a good and accurate fine-tuned model.

The hardware

I bought a desktop machine for work recently. It has an AMD Ryzen 9 9950X CPU, 64GB of DDR5 RAM and 4 TB of SSD storage. The most critical component though is the Nvidia GeForce RTX 5090 GPU which has 32GB of VRAM and memory bandwidth of

1,792 GB/s. This is the machine that I wanted to try to fine-tune Llama-3.1–8B-Instruct on.

If you have the suspicion that this fine-tuning project is just an excuse to take the machine for a run, you are absolutely correct.

At the Nvidia GTC 2025 where I bought the Nvidia GeForce 5090, with colleagues

Choosing the tool

Fine-tuning is not so new anymore and there are a few ways of doing it. The most compatible way is probably just using Huggingface Transformers and other Huggingface libraries. However that's also not the fastest and with a single desktop GPU, I would prefer to have a faster way of doing it (actually regardless, I'd want try to fine-tune faster than slower).

The other popular ways are to use Unsloth or Axolotl. Both are open source tools that can speed up fine-tuning open weight models like Llama-3.1–8B-Instruct and both are built on top of Huggingface Transformers.

Unsloth is a performance-optimized wrapper around Hugging Face Transformers that accelerates LLM fine-tuning by up to 2x, while significantly reducing GPU memory usage. It is ideal for instruction tuning or continued pretraining on consumer GPUs.

Axolotl on the other hand is a highly configurable fine-tuning framework also built on top of Hugging Face's ecosystem, designed for complex, large-scale training pipelines.

Just from the sound of it, it seems Unsloth is probably the better tool to use, but the deciding factor is that Axolotl doesn't support RTX 5090! So Unsloth it is.

Setting up the environment

Unsurprisingly, but also surprisingly for me, the machine and the fine-tuning software are very much intertwined. I've been spending so much time on platforms where the software I write are portable from one machine to another and even from operating system to operating system, having to deal with quirks of individual machines are both interesting and annoying at the same time.

The main issue here is the RTX 5090 GPU.

The GeForce RTX 5090 is a Blackwell–generation card whose compute capability is *sm_120* (CUDA 12.x). At this point in time (July 2025) PyTorch latest stable version is 2.7.1 and it does support CUDA 12.x but the default installation only supports up to 12.6 while the RTX 5090 needs CUDA 12.8.

To install the libraries it's best to use uv.

```
curl -LsSf https://astral.sh/uv/install.sh | sh && source $HOME/.local/bin/env
```

Then create your virtual environment (or use conda, if that's what you're used to) with Python 3.12:

```
uv venv .venv --python=3.12 --seed
source .venv/bin/activate
```

Now you can install the correct PyTorch version, but remember you need to specify the wheel as well:

```
uv pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cu128
```

That was probably the biggest stumbling block so far and getting around PyTorch compatibility can be a nightmare. To make matters worse, I was trying it out before the current stable version and I had to even go use the nightly version of PyTorch 2.8 to make it work. It took me a while to figure it out.

Besides Pytorch, there are a bunch of other libraries we need to run Unsloth.

```
uv pip install unsloth unsloth_zoo bitsandbytes
```

The trickiest amongst them (for me) is Triton. As of now, you need to use `triton>=3.3.1` or else Unsloth will not run smoothly. If you're thinking of just not using Triton and disabling it, this doesn't work very well because parts of Unsloth itself calls Triton and if it is wrong version, it can't work with RTX 5090.

```
uv pip install triton>=3.3.1
```

You will of course need Huggingface Transformers but you need to use a specific version.

```
uv pip install -U transformers==4.52.4
```

Getting the base model and using Weights and Biases

We also need log into Huggingface and Weights and Biases.

Huggingface because we need to download the base model from which we'll fine-tune. Why do we need this? It's because Llama-3.1–8B-Instruct is a restricted model which you need to log into Huggingface in order to load and use for fine-tuning.

Just log into https://huggingface.co/ (and if you don't have an account, just create one) and then go to https://huggingface.co/settings/tokens. Create a user access token. Then go back to your terminal and set the token as an environment variable.

```
$ export HF_TOKEN=<your user access token>
```

As for Weights and Biases, we use it to do logging but it has a lot of other capabilities. It's a very useful utility that helps us capture the logs (amongst other things) so we can do other stuff and be able to monitor without logging in and checking the logs

ourselves.

Register an account and login at https://wandb.ai/ and then go to https://wandb.ai/authorize to get the API key. Then you can go to the command line and run this:

```
$ wandb login
```

and paste the API key there. Alternatively you can also set the `WANDB_API_KEY` environment variable before running the script.

The script

This is a big long (600+ lines) but it's the entire script to fine-tune Llama-3.1–8B-Instruct.

```
#!/usr/bin/env python3
"""
Fine-tune Llama 3.1 8B Instruct on Singapore Legislation
using Unsloth with PEFT/LoRA
Optimized for RTX 5090 (Blackwell) with CUDA 12.8

Hardware Requirements:
- AMD Ryzen 9 9950X, 64GB RAM, RTX-5090 GPU (Blackwell)
- OS: Ubuntu 24.04.2 LTS
- CUDA: 12.8
"""

import os
import sys
import logging
import math
import json
from datetime import datetime
from datasets import load_from_disk, Dataset

import torch
import numpy as np
from unsloth import FastLanguageModel
from trl import SFTTrainer
from transformers import (
    TrainingArguments,
    TextStreamer,
    TrainerCallback,
    EarlyStoppingCallback,
    TrainerControl,
    TrainerState,
)
from peft import LoraConfig
import wandb
import torch.nn.functional as F
from typing import Optional, Union, Dict, Any, List, Tuple


class RTX5090OptimizedTrainer(SFTTrainer):
    """Custom SFT trainer for RTX 5090 with stability improvements."""

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        """
        Stable loss computation with Triton acceleration when available.
        Optimized for RTX 5090 with enhanced performance.
        """
        # Get model outputs
        outputs = model(
            input_ids=inputs.get("input_ids"),
            attention_mask=inputs.get("attention_mask"),
            labels=inputs.get("labels"),
            return_dict=True,
        )

        # Get logits and labels
        logits = outputs.logits
        labels = inputs.get("labels")

        # Shift so that tokens < n predict n
```

```
        shift_logits = logits[..., :-1, :].contiguous()
        shift_labels = labels[..., 1:].contiguous()

        # Flatten the tokens
        loss_fct = torch.nn.CrossEntropyLoss(ignore_index=-100)
        shift_logits = shift_logits.view(-1, shift_logits.size(-1))
        shift_labels = shift_labels.view(-1)

        # Calculate loss with numerical stability
        loss = loss_fct(shift_logits, shift_labels)

        # Check for NaN/Inf and handle gracefully
        if torch.isnan(loss) or torch.isinf(loss):
            logger.warning(
                "NaN/Inf loss detected, using fallback computation"
            )
            loss = torch.tensor(
                0.0, device=loss.device, requires_grad=True
            )

        return (loss, outputs) if return_outputs else loss


class StabilityCallback(TrainerCallback):
    """Callback for training stability and divergence detection."""

    def __init__(
        self,
        divergence_threshold=1.3,
        min_steps_before_check=100,
        patience=2,
        no_improvement_patience=5,
        gradient_explosion_threshold=5.0,
    ):
        super().__init__()
        self.divergence_threshold = divergence_threshold
        self.min_steps_before_check = min_steps_before_check
        self.patience = patience
        self.no_improvement_patience = no_improvement_patience
        self.gradient_explosion_threshold = gradient_explosion_threshold
        self.best_val_loss = float("inf")
        self.divergence_count = 0
        self.no_improvement_count = 0
        self.validation_losses = []
        self.gradient_norms = []
        self.consecutive_high_gradients = 0

    def on_log(self, args, state, control, logs=None, **kwargs):
        """Monitor gradient norms and training stability."""
        if logs:
            # Track gradient norms
            if "grad_norm" in logs:
                self.gradient_norms.append(logs["grad_norm"])

                # Check for gradient explosion
                if logs["grad_norm"] > self.gradient_explosion_threshold:
                    self.consecutive_high_gradients += 1
                    logger.error(
                        f"Gradient explosion detected: {
                            logs['grad_norm']:.4f} > {
                            self.gradient_explosion_threshold}"
                    )

                    if self.consecutive_high_gradients >= 3:
                        logger.error(
                            (
                            "Stopping training due to persistent "
                            "gradient explosion"
                        )
                        )
                        control.should_training_stop = True
                        return
                elif logs["grad_norm"] > 1.0:
                    logger.warning(
                        f"High gradient norm detected: {
                            logs['grad_norm']:.4f}"
                    )
                    self.consecutive_high_gradients = max(
                        0, self.consecutive_high_gradients - 1
                    )
```

```
                else:
                    self.consecutive_high_gradients = 0

                # Log gradient statistics
                if len(self.gradient_norms) >= 10:
                    recent_norms = self.gradient_norms[-10:]
                    avg_norm = np.mean(recent_norms)
                    max_norm = np.max(recent_norms)

                    wandb.log(
                        {
                            "gradient_norm_avg_10": avg_norm,
                            "gradient_norm_max_10": max_norm,
                            "gradient_norm_current": logs["grad_norm"],
                            "consecutive_high_gradients":
                                self.consecutive_high_gradients,
                        },
                        step=state.global_step,
                    )

            # Check for NaN/Inf in training loss
            if "train_loss" in logs:
                if np.isnan(logs["train_loss"]) or np.isinf(
                    logs["train_loss"]
                ):
                    logger.error(
                        (
                        f"NaN/Inf training loss detected at "
                        f"step {state.global_step}"
                    )
                    )
                    control.should_training_stop = True

    def on_evaluate(self, args, state, control, logs=None, **kwargs):
        """Callback for divergence detection and early stopping."""
        if (
            logs
            and "eval_loss" in logs
            and state.global_step >= self.min_steps_before_check
        ):
            current_val_loss = logs["eval_loss"]
            self.validation_losses.append(current_val_loss)

            # Check for improvement
            if current_val_loss < self.best_val_loss:
                improvement = self.best_val_loss - current_val_loss
                self.best_val_loss = current_val_loss
                self.divergence_count = 0
                self.no_improvement_count = 0
                logger.info(
                    f"New best validation loss: {
                        self.best_val_loss:.4f} (improvement: {
                        improvement:.4f})"
                )
            else:
                self.no_improvement_count += 1

                # Check for divergence
                if (
                    current_val_loss
                    > self.best_val_loss * self.divergence_threshold
                ):
                    self.divergence_count += 1
                    logger.warning(
                        f"Potential divergence detected: {
                            current_val_loss:.4f} > {
                            self.best_val_loss *
                            self.divergence_threshold:.4f} (count: {
                            self.divergence_count})"
                    )

                    if self.divergence_count >= self.patience:
                        logger.error(
                            f"Training diverged! Stopping at step {
                                state.global_step}"
                        )
                        control.should_training_stop = True
                        return

                # Check for no improvement
```

```
                if (
                    self.no_improvement_count
                    >= self.no_improvement_patience
                ):
                    logger.warning(
                        (
                        (f"No improvement for {self.no_improvement_count} "
                         f"evaluations. Stopping training.")
                    )
                    )
                    control.should_training_stop = True
                    return

            # Calculate perplexity
            perplexity = math.exp(current_val_loss)

            # Enhanced logging
            wandb.log(
                {
                    "eval_perplexity": perplexity,
                    "best_val_loss": self.best_val_loss,
                    "divergence_count": self.divergence_count,
                    "no_improvement_count": self.no_improvement_count,
                    "val_loss_trend": (
                        current_val_loss - self.validation_losses[-2]
                        if len(self.validation_losses) >= 2
                        else 0
                    ),
                },
                step=state.global_step,
            )

            logger.info(
                f"Step {
                    state.global_step}: Validation Loss: {
                    current_val_loss:.4f}, Perplexity: {
                    perplexity:.2f}, No improvement: {
                    self.no_improvement_count}"
            )


def print_system_info():
    """System information."""
    logger.info("=== System Information ===")
    logger.info(f"Python version: {sys.version}")
    logger.info(f"PyTorch version: {torch.__version__}")
    logger.info(f"CUDA available: {torch.cuda.is_available()}")

    if torch.cuda.is_available():
        logger.info(f"CUDA version: {torch.version.cuda}")
        logger.info(f"GPU count: {torch.cuda.device_count()}")
        logger.info(f"GPU name: {torch.cuda.get_device_name(0)}")
        logger.info(
            f"GPU memory: {
                torch.cuda.get_device_properties(0).total_memory /
                1024**3:.1f} GB"
        )

    logger.info("=== End System Information ===")


def format_instruction_chat(example):
    """Format examples for instruction tuning with chat template."""
    # Extract the content from the example
    content = example.get("content", example.get("text", ""))

    # Create instruction-response format for legal documents
    instruction = (
        "You are a helpful assistant specialized in Singapore "
        "legislation. Please provide accurate information about the "
        "following legal document."
    )

    # Format as chat conversation
    parts = [
        "<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\n",
        instruction,
        "<|eot_id|><|start_header_id|>user<|end_header_id|>\n\n",
        "Please explain this Singapore legislation:<|eot_id|>",
        "<|start_header_id|>assistant<|end_header_id|>\n\n",
```

```
        content,
        "<|eot_id|>",
    ]
    formatted_text = "".join(parts)

    return {"text": formatted_text}


def load_and_prepare_dataset(dataset_path):
    """Load and prepare the Singapore legislation dataset."""
    logger.info(f"Loading dataset from {dataset_path}")

    try:
        dataset = load_from_disk(dataset_path)
        logger.info(f"Dataset loaded successfully: {dataset}")

        # Apply chat formatting
        logger.info("Applying instruction formatting...")
        dataset = dataset.map(
            format_instruction_chat,
            remove_columns=dataset["train"].column_names,
        )

        # Verify dataset structure
        logger.info(
            (
            f"Sample formatted text: "
            f"{dataset['train'][0]['text'][:120]}..."
        )
        )

        return dataset

    except Exception as e:
        logger.error(f"Failed to load dataset: {e}")
        raise


def find_latest_checkpoint(output_dir):
    """Find the latest checkpoint in the output directory."""
    if not os.path.exists(output_dir):
        return None

    checkpoints = []
    for item in os.listdir(output_dir):
        if item.startswith("checkpoint-") and os.path.isdir(
            os.path.join(output_dir, item)
        ):
            try:
                step_num = int(item.split("-")[1])
                checkpoints.append(
                    (step_num, os.path.join(output_dir, item))
                )
            except (ValueError, IndexError):
                continue

    if checkpoints:
        # Return the checkpoint with the highest step number
        latest_step, latest_path = max(checkpoints, key=lambda x: x[0])
        logger.info(
            f"Found latest checkpoint: {latest_path} (step {latest_step})"
        )
        return latest_path

    return None


def main():
    # Fixed parameters - Conservative settings based on lessons learned
    # Model and dataset configuration
    model_name = "meta-llama/Meta-Llama-3.1-8B-Instruct"
    dataset_path = "./hf_dataset/sg_legislation_instructions"
    output_dir = "./llama-3.1-8b-instruct-sg-legislation"

    # Training parameters - Proven stable hyperparameters from memory
    learning_rate = 2e-5  # Conservative learning rate for stability
    num_epochs = 2
    batch_size = 4
    gradient_accumulation_steps = 2
    max_seq_length = 2048
```

```
    warmup_steps = 500
    weight_decay = 0.1  # Strong regularization
    max_grad_norm = 0.3  # Conservative gradient clipping

    # LoRA parameters - Reduced for stability
    lora_r = 16  # Reduced rank for stability
    lora_alpha = 16
    lora_dropout = 0.2

    # Early stopping and stability
    early_stopping_patience = 4
    eval_steps = 250
    save_steps = 500

    # Weights & Biases
    wandb_project = "llama-3.1-8b-instruct-sg-legislation"
    wandb_run_name = None

    # Other parameters
    resume_from_checkpoint = None
    seed = 42

    # Setup logging
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s",
        handlers=[
            logging.FileHandler("finetune_llama.log"),
            logging.StreamHandler(sys.stdout),
        ],
    )

    global logger
    logger = logging.getLogger(__name__)

    # Print system information
    print_system_info()

    # Set random seeds
    torch.manual_seed(seed)
    np.random.seed(seed)

    # Initialize Weights & Biases
    run_name = (
        wandb_run_name
        or f"llama-{datetime.now().strftime('%Y%m%d-%H%M%S')}"
    )
    wandb.init(
        project=wandb_project,
        name=run_name,
        config={
            "model_name": model_name,
            "dataset_path": dataset_path,
            "learning_rate": learning_rate,
            "num_epochs": num_epochs,
            "batch_size": batch_size,
            "gradient_accumulation_steps": gradient_accumulation_steps,
            "max_seq_length": max_seq_length,
            "warmup_steps": warmup_steps,
            "weight_decay": weight_decay,
            "max_grad_norm": max_grad_norm,
            "lora_r": lora_r,
            "lora_alpha": lora_alpha,
            "lora_dropout": lora_dropout,
            "early_stopping_patience": early_stopping_patience,
            "seed": seed,
        },
    )

    logger.info(
        "Starting Llama 3.1 8B Instruct fine-tuning for RTX 5090..."
    )
    logger.info(
        (f"Fixed parameters: model={model_name}, lr={learning_rate}, "
f"epochs={num_epochs}")
    )

    # Auto-detect checkpoint if not specified
    if resume_from_checkpoint is None:
        latest_checkpoint = find_latest_checkpoint(output_dir)
```

```
        if latest_checkpoint:
            logger.info(
                (
                f"Auto-resuming from latest checkpoint: "
                f"{latest_checkpoint}"
            )
            )
            resume_from_checkpoint = latest_checkpoint
        else:
            logger.info(
                "No existing checkpoints found. Starting fresh training."
            )
    elif resume_from_checkpoint and not os.path.exists(
        resume_from_checkpoint
    ):
        logger.warning(
            (f"Checkpoint {resume_from_checkpoint} not found. "
"Starting fresh training.")
        )
        resume_from_checkpoint = None

    # Load dataset
    dataset = load_and_prepare_dataset(dataset_path)

    # Load model and tokenizer with Unsloth optimizations
    logger.info("Loading model and tokenizer with Unsloth...")
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name=model_name,
        max_seq_length=max_seq_length,
        dtype=None,  # Auto-detect
        load_in_4bit=True,  # Use 4-bit quantization for RTX 5090
        device_map="auto",
    )

    # Configure LoRA
    model = FastLanguageModel.get_peft_model(
        model,
        r=lora_r,
        target_modules=[
            "q_proj",
            "k_proj",
            "v_proj",
            "o_proj",
            "gate_proj",
            "up_proj",
            "down_proj",
        ],
        lora_alpha=lora_alpha,
        lora_dropout=lora_dropout,
        bias="none",
        use_gradient_checkpointing="unsloth",  # Unsloth's checkpointing
        random_state=seed,
    )

    # Setup training arguments
    training_args = TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=num_epochs,
        per_device_train_batch_size=batch_size,
        per_device_eval_batch_size=batch_size,
        gradient_accumulation_steps=gradient_accumulation_steps,
        learning_rate=learning_rate,
        weight_decay=weight_decay,
        max_grad_norm=max_grad_norm,
        warmup_steps=warmup_steps,
        lr_scheduler_type="cosine",  # Stable cosine scheduler
        logging_steps=50,
        eval_steps=eval_steps,
        save_steps=save_steps,
        eval_strategy="steps",  # Fixed: was evaluation_strategy
        save_strategy="steps",
        load_best_model_at_end=True,
        metric_for_best_model="eval_loss",
        greater_is_better=False,
        report_to="wandb",
        run_name=run_name,
        seed=seed,
        data_seed=seed,
        # RTX 5090 optimizations
        bf16=True,  # Use bf16 to match model precision
```

```
    fp16=False,
    dataloader_pin_memory=True,
    dataloader_num_workers=4,
    remove_unused_columns=False,
    # Stability improvements
    save_safetensors=True,
    ddp_find_unused_parameters=False,
)

# Initialize callbacks with enhanced stability monitoring
stability_callback = StabilityCallback(
    divergence_threshold=1.3,  # More conservative threshold
    min_steps_before_check=100,  # Check earlier
    patience=2,  # Less patience for divergence
    no_improvement_patience=5,  # Stop if no improvement
    gradient_explosion_threshold=5.0,  # Stop if gradients exceed 5.0
)

early_stopping_callback = EarlyStoppingCallback(
    early_stopping_patience=early_stopping_patience,
)

# Initialize trainer
logger.info("Initializing RTX 5090 optimized trainer...")
trainer = RTX5090OptimizedTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    tokenizer=tokenizer,
    packing=True,
    dataset_kwargs={
        "add_special_tokens": False,
        "append_concat_token": False,
    },
    callbacks=[stability_callback, early_stopping_callback],
)

# Log training information
logger.info(f"Training samples: {len(dataset['train'])}")
logger.info(f"Validation samples: {len(dataset['validation'])}")
logger.info(
    f"Effective batch size: {batch_size * gradient_accumulation_steps}"
)

# Start training
logger.info("Starting training...")
try:
    trainer.train(resume_from_checkpoint=resume_from_checkpoint)
    logger.info("Training completed successfully!")
except Exception as e:
    logger.error(f"Training failed: {e}")
    raise

# Save the model
logger.info("Saving model...")
trainer.save_model(output_dir)
tokenizer.save_pretrained(output_dir)

# Save merged model with Unsloth
logger.info("Saving merged model...")
try:
    model.save_pretrained_merged(
        output_dir,
        tokenizer,
        save_method="merged_16bit",
    )
    logger.info("Merged model saved successfully!")
except Exception as e:
    logger.warning(f"Failed to save merged model: {e}")

# Save training configuration
config_path = os.path.join(output_dir, "training_config.json")
with open(config_path, "w") as f:
    json.dump(
        {
            "model_name": model_name,
            "dataset_path": dataset_path,
```

```
                "training_args": training_args.to_dict(),
                "lora_config": {
                    "r": lora_r,
                    "alpha": lora_alpha,
                    "dropout": lora_dropout,
                },
                "environment": {
                    "torch_version": torch.__version__,
                    "cuda_version": torch.version.cuda,
                    "gpu_name": (
                        torch.cuda.get_device_name(0)
                        if torch.cuda.is_available()
                        else "N/A"
                    ),
                    "optimized_for": "RTX-5090-Blackwell",
                },
            },
            f,
            indent=2,
        )

    logger.info(f"Model saved to {output_dir}")
    logger.info("Fine-tuning completed successfully!")

    # Finish wandb
    wandb.finish()


if __name__ == "__main__":
    main()
```

The script uses a number of libraries, primarily Unsloth and its fast model, Huggingface Transformers and PEFT for LoRA support and TRL for supervised training with SFTTrainer. It also uses wandb for experiment tracking and the datasets library to load and manage datasets from disk.

I have a few convenience functions including `find_latest_checkpoint` for automatically resuming from the latest checkpoint, `format_instruction_chat` for formatting raw examples into instruction-response chat turns, and `load_and_prepare_dataset()` for loading the dataset. Finally I have `print_system_info` for printing system specifications to the screen on startup. These abstractions keep the main script clean.

Two important classes extend the training framework. The first, `RTX5090OptimizedTrainer`, subclasses `SFTTrainer` to override the loss computation, ensuring that any unstable values (`NaNs` or `Infs`) are caught and zeroed out rather than crashing training. The second, `StabilityCallback`, monitors training health through mechanisms like divergence checks, early stopping based on stagnation, and gradient norm explosion detection. It works alongside Huggingface's built-in `EarlyStoppingCallback` to make training robust even in long or fragile fine-tunes.

The `main` function orchestrates the entire fine-tuning process. Hyper-parameters are declared early and logged to wandb. Logging is configured, seeds are set for reproducibility, and system info is printed to both console and file. If checkpointing is enabled or necessary, the script automatically resumes from the latest saved state.

Here's the list of hyper-parameters used:

- **learning_rate = 2e-5**
  A conservative learning rate that controls how quickly the model updates during training.
- **num_epochs = 2**
  The number of full passes through the training dataset.
- **batch_size = 4**
  Number of examples processed simultaneously per training step.
- **gradient_accumulation_steps = 2**
  Accumulates gradients over two steps before updating weights, effectively simulating a batch size of 8.
- **max_seq_length = 2048**
  Maximum number of tokens per training example (input + output combined).

- **warmup_steps = 500**
  Number of steps to gradually increase the learning rate from zero to its target value.
- **weight_decay = 0.1**
  L2 regularization to prevent overfitting by penalizing large weights.
- **max_grad_norm = 0.3**
  Gradient clipping to prevent unstable updates or exploding gradients.
- **lora_r = 16**
  The rank (dimensionality) of the low-rank adaptation matrices — smaller values reduce memory and improve stability.
- **lora_alpha = 16**
  A scaling factor that adjusts the strength of the LoRA update relative to the base model.
- **lora_dropout = 0.2**
  Dropout rate applied to LoRA adapters to reduce overfitting.

Next, the dataset is loaded from disk, reformatted into instruction-style prompts suitable for chat models, and split into train and validation sets. The model is then loaded via `FastLanguageModel.from_pretrained` from Unsloth, which applies 4-bit quantization and automatically maps the model to available GPU memory. LoRA adapters are injected using `get_peft_model`, targeting projection matrices such as `q_proj`, `v_proj`, and `gate_proj`.

Training arguments are set up with Huggingface's `TrainingArguments`, including configurations for mixed-precision (`bfloat16`), cosine learning rate decay, frequent evaluation steps, and automatic checkpointing. The custom `RTX5090OptimizedTrainer` is then initialized with the model, datasets, tokenizer, and callbacks. Training is kicked off using `trainer.train`, and if enabled, the model will automatically reload the best checkpoint based on validation loss.

After training completes, a few things happen. The `trainer.save_model` method captures the fine-tuning session and stores the lightweight adapter weights, preserving the separation between these task-specific parameters and the untouched base model.

Then, `tokenizer.save_pretrained` saves the tokenizer assets — its vocabulary, configuration, and any added special tokens — so the inference pipeline can recreate the exact tokenization environment later. This creates `tokenizer.json`, `tokenizer_config.json`, and `special_tokens_map.json`.

Next, `model.save_pretrained_merged` fuses the base and adapter weights and exports the result in 16-bit precision, producing a fully self-contained model ready for deployment or sharing.

Finally, all the configuration is dumped into `training_config.json`.

Running the script

Now that we have the script let's run it. You really want to run the script in the background especially if you are SSH'ing into the machine and you don't want to disconnect in the middle of the fine-tuning and you end up restarting everything.

There are many ways of doing this but the easiest is probably to use `tmux`. If it's not already installed, just install it normally and then create a new session.

```
$ tmux new -s finetune
```

Now you can run the script.

```
$ python finetune_llama.py
```

At any point in time you can detach from the session by pressing ctrl-b then d. If you want to reattach back to the session, you can run this command:

```
$ tmux attach -t finetune
```

Here's the start of the output of the script when you run it.

```
sausheong@riptide$ source venv/bin/activate
(venv) sausheong@riptide$ python finetune_llama.py
🦥 Unsloth: Will patch your computer to enable 2x faster free finetuning.
🦥 Unsloth Zoo will now patch everything to make training faster!
INFO 07-13 11:23:31 [__init__.py:244] Automatically detected platform cuda.
2025-07-13 11:23:32,919 - INFO - === System Information ===
2025-07-13 11:23:32,919 - INFO - Python version: 3.12.9 | packaged by Anaconda, Inc. | (main, Feb  6 2025, 18:56:27) [GCC 1
2025-07-13 11:23:32,919 - INFO - PyTorch version: 2.7.0+cu128
2025-07-13 11:23:32,919 - INFO - CUDA available: True
2025-07-13 11:23:32,919 - INFO - CUDA version: 12.8
2025-07-13 11:23:32,919 - INFO - GPU count: 1
2025-07-13 11:23:32,920 - INFO - GPU name: NVIDIA GeForce RTX 5090
2025-07-13 11:23:32,920 - INFO - GPU memory: 31.4 GB
2025-07-13 11:23:32,920 - INFO - === End System Information ===
wandb: Currently logged in as: sausheong (sausheongchang) to https://api.wandb.ai. Use `wandb login --relogin` to force rel
wandb: Tracking run with wandb version 0.21.0
wandb: Run data is saved locally in /home/sausheong/finetune_sg_legislation/wandb/run-20250713_112333-70utt9lj
wandb: Run `wandb offline` to turn off syncing.
wandb: Syncing run llama-3.1-rtx5090-20250713-112332
wandb: ⭐️ View project at https://wandb.ai/sausheongchang/llama-3.1-sg-legislation
wandb: 🚀 View run at https://wandb.ai/sausheongchang/llama/runs/70utt9lj
2025-07-13 11:23:34,495 - INFO - Starting Llama 3.1 8B Instruct fine-tuning for RTX 5090...
2025-07-13 11:23:34,495 - INFO - Fixed parameters: model=meta-llama/Meta-Llama-3.1-8B-Instruct, lr=2e-05, epochs=2
2025-07-13 11:23:34,495 - INFO - No existing checkpoints found. Starting fresh training.
2025-07-13 11:23:34,495 - INFO - Loading dataset from ./hf_dataset/sg_legislation_instructions
2025-07-13 11:23:34,501 - INFO - Dataset loaded successfully: DatasetDict({
    train: Dataset({
        features: ['instruction', 'input', 'output'],
        num_rows: 177832
    })
    validation: Dataset({
        features: ['instruction', 'input', 'output'],
        num_rows: 22229
    })
    test: Dataset({
        features: ['instruction', 'input', 'output'],
        num_rows: 22229
    })
})
2025-07-13 11:23:34,501 - INFO - Applying instruction formatting...
Map: 100%|████████████████████████████████████████████████████████████████████████████████| 177832/1778
Map: 100%|████████████████████████████████████████████████████████████████████████████████| 22229/222
Map: 100%|████████████████████████████████████████████████████████████████████████████████| 22229/222
2025-07-13 11:23:36,126 - INFO - Sample formatted text: <|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant specialized in Singapore legisl...
2025-07-13 11:23:36,126 - INFO - Loading model and tokenizer with Unsloth...
==((====))==  Unsloth 2025.6.12: Fast Llama patching. Transformers: 4.52.4. vLLM: 0.9.2rc2.dev29+gf73d02aad.
   \\   /|    NVIDIA GeForce RTX 5090. Num GPUs = 1. Max memory: 31.367 GB. Platform: Linux.
O^O/ \_/ \    Torch: 2.7.0+cu128. CUDA: 12.0. CUDA Toolkit: 12.8. Triton: 3.3.1
\        /    Bfloat16 = TRUE. FA [Xformers = None. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
2025-07-13 11:23:41,423 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to
Unsloth: Dropout = 0 is supported for fast patching. You are using dropout = 0.2.
Unsloth will patch all other layers, except LoRA matrices, causing a performance hit.
Unsloth 2025.6.12 patched 32 layers with 0 QKV layers, 0 O layers and 0 MLP layers.
2025-07-13 11:23:50,215 - INFO - Initializing RTX 5090 optimized trainer...
Unsloth: Tokenizing ["text"]: 100%|████████████████████████████████████████████████████████| 177832/177
Unsloth: Tokenizing ["text"]: 100%|████████████████████████████████████████████████████████| 22229/22
2025-07-13 11:23:53,202 - INFO - Training samples: 177832
2025-07-13 11:23:53,202 - INFO - Validation samples: 22229
2025-07-13 11:23:53,202 - INFO - Effective batch size: 8
2025-07-13 11:23:53,202 - INFO - Starting training...
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 177,832 | Num Epochs = 2 | Total steps = 44,458
O^O/ \_/ \    Batch size per device = 4 | Gradient accumulation steps = 2
\        /    Data Parallel GPUs = 1 | Total batch size (4 x 2 x 1) = 8
 "-____-"     Trainable parameters = 41,943,040 of 8,000,000,000 (0.52% trained)
```

```
  0%|
  0%|
```

Waiting for the fine-tuning to complete

Now sit back and relax (sort of). You can check every now and then to see what's happening, by reattaching back to the tmux session but you can also just log into https://www.wandb.ai and the go to your project and then open up the visualisations or logs. You can literally start your fine-tuning and then go get lunch or do something else, and then monitor what's happening at the machine. It can take hours or days, but for this case, it took me about 6 hours or so to finish.

When the fine-tuning completes your output directory should look something like this (along with a bunch of checkpoint directories):

```
(venv) sausheong@riptide$ ls -lh
...
-rw-rw-r--  1 sausheong sausheong 1008 Jul 13 13:42 config.json
-rw-rw-r--  1 sausheong sausheong  234 Jul 13 13:42 generation_config.json
-rw-rw-r--  1 sausheong sausheong 4.7G Jul 13 13:44 model-00001-of-00004.safetensors
-rw-rw-r--  1 sausheong sausheong 4.7G Jul 13 13:46 model-00002-of-00004.safetensors
-rw-rw-r--  1 sausheong sausheong 4.6G Jul 13 13:48 model-00003-of-00004.safetensors
-rw-rw-r--  1 sausheong sausheong 1.1G Jul 13 13:48 model-00004-of-00004.safetensors
-rw-rw-r--  1 sausheong sausheong  24K Jul 13 13:42 model.safetensors.index.json
-rw-rw-r--  1 sausheong sausheong 1.9K Jul 13 13:42 README.md
-rw-rw-r--  1 sausheong sausheong  454 Jul 13 13:42 special_tokens_map.json
-rw-rw-r--  1 sausheong sausheong  50K Jul 13 13:42 tokenizer_config.json
-rw-rw-r--  1 sausheong sausheong  17M Jul 13 13:42 tokenizer.json
-rw-rw-r--  1 sausheong sausheong 6.1K Jul 13 13:42 training_args.bin
-rw-rw-r--  1 sausheong sausheong 4.7K Jul 13 13:48 training_config.json
```

The `config.json` file stores the model architecture details, such as the number of layers, hidden size, and attention heads. This is critical for reconstructing the model during inference. The full merged model weights are spread across four files — `model-00001-of-00004.safetensors` through `model-00004-of-00004.safetensors`. These files hold the trained parameters and are referenced by the `model.safetensors.index.json`, which acts as a pointer to the sharded weight files and allows the model loader (like Huggingface's `AutoModelForCausalLM`) to know how to load them correctly.

Tokenization is also a key part of inference, and several files handle that. The `tokenizer.json` contains the serialized tokenizer vocabulary and its internal structure. It's complemented by `tokenizer_config.json`, which specifies tokenizer settings such as whether the text should be lowercased and what tokenizer class to use. The `special_tokens_map.json` maps key tokens like BOS (beginning-of-sequence), EOS (end-of-sequence), and PAD (padding) to their token IDs so the tokenizer and model can handle them appropriately. An optional but helpful file is `generation_config.json`, which stores default text generation parameters like temperature, top-k, and top-p. While not strictly required, it ensures consistent behavior when generating text.

The remaining files are not necessary for inference. The `README.md` typically documents the model's purpose, usage, and any important notes. The `training_args.bin` is a serialized version of the training arguments (from Huggingface's TrainingArguments), useful for retraining or verifying training settings. Similarly, `training_config.json` contains a readable version of the hyperparameters used, as mentioned before. These are all useful for documentation or re-running training but are not needed for inference.

What could go wrong?

This is of course, if everything is ideal. Plenty of things can go wrong during fine-tuning.

One of the most common sources of issues during fine-tuning stems from the data itself. Poor data quality — such as inconsistent formatting, uncleaned text, or mislabeled examples — can confuse the model and hinder its ability to learn effectively. If the dataset format does not match the model's expected input-output structure (as in instruction tuning), training may proceed without errors but yield unusable results.

Sequence lengths that exceed the model's maximum token limit can lead to silent truncation or outright crashes. Additionally, using too little data may result in underfitting, where the model fails to generalize. Another common but serious issue is data leakage, where evaluation sets overlap with training data, producing misleadingly high performance metrics.

Training configuration is another frequent source of problems. An excessively high learning rate can cause training loss to spike or diverge entirely, while a learning rate that is too low may result in training stagnation.

Inappropriate batch sizes can also cause instability: large batches may cause GPU memory to overflow, and very small ones can make training noisy or inefficient.

Overfitting is a risk when training for too long or on small datasets — the model performs well on the training set but poorly on unseen data. On the other hand, underfitting occurs when the model capacity or training duration is insufficient to capture the patterns in the data, leading to consistently high loss on both training and validation sets.

Hardware and environment-related issues can also derail training. The most common is running out of GPU memory, especially when fine-tuning large models or using long sequences. Using the wrong CUDA version, or an incompatible PyTorch build, may lead to initialization failures or runtime errors. Even less obvious problems — such as GPU thermal throttling — can slow training dramatically without any explicit warning.

All these makes the ability to resume from checkpoints quite critical. You don't want to have to start from scratch every time you face an issue.

Inferencing with the model

At the end of the fine-tuning, the script merged the adapter with the base model, and so we only really need to create a normal inference script. Here's a simple one to use our merged model.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, TextStreamer
import argparse
import readline


def load_model(model_path, device):
    """
    Load the model and tokenizer from the specified path.
    """
    print(f"Loading model from {model_path}...")
    tokenizer = AutoTokenizer.from_pretrained(model_path)

    # For CPU, we might need to load in a different precision
    torch_dtype = (
        torch.bfloat16
        if device == "cuda" and torch.cuda.is_bf16_supported()
        else torch.float16
    )

    model = AutoModelForCausalLM.from_pretrained(
        model_path,
        torch_dtype=torch_dtype,
        device_map="auto",  # Let transformers handle device placement
    )
    print("Model loaded successfully.")
    return model, tokenizer


def format_prompt(prompt):
    """
    Format the prompt using the Llama 3 instruction format.
    """
    messages = [
        {
            "role": "system",
            "content": (
                "You are a helpful assistant knowledgeable in "
                "Singaporean law."
            ),
        },
        {"role": "user", "content": prompt},
    ]

    # Use the tokenizer's chat template for the most robust formatting
    formatted_prompt = AutoTokenizer.from_pretrained(
        "./llama-3.1-8b-instruct-sg-legislation/checkpoint-2500"
    ).apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
```

```
    )
    return formatted_prompt


def generate_response(
    model, tokenizer, prompt, device, max_new_tokens=512, temperature=0.7
):
    """
    Generate a response from the model.
    """
    formatted_prompt = format_prompt(prompt)
    inputs = tokenizer(formatted_prompt, return_tensors="pt").to(
        model.device
    )

    streamer = TextStreamer(
        tokenizer, skip_prompt=True, skip_special_tokens=True
    )

    # Generate the response
    _ = model.generate(
        **inputs,
        streamer=streamer,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        temperature=temperature,
        # Stop generation when these tokens are encountered.
        eos_token_id=[
            tokenizer.eos_token_id,
            tokenizer.convert_tokens_to_ids("<|eot_id|>"),
        ],
        pad_token_id=tokenizer.eos_token_id,
    )
    return ""  # Streamer handles the output


def main():
    """
    Main function to run the inference script.
    """
    parser = argparse.ArgumentParser(
        description="Inference script for fine-tuned Llama 3.1 model."
    )
    parser.add_argument(
        "--model_path",
        type=str,
        default="./llama-3.1-8b-instruct-sg-legislation",
        help="Path to the fine-tuned model.",
    )
    parser.add_argument(
        "--prompt", type=str, default=None, help="A single prompt to run."
    )
    parser.add_argument(
        "--max_new_tokens",
        type=int,
        default=1024,
        help="Maximum number of new tokens to generate.",
    )
    parser.add_argument(
        "--temperature",
        type=float,
        default=0.7,
        help="Temperature for sampling.",
    )
    args = parser.parse_args()

    # Determine the device
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Using device: {device}")

    # Load the model and tokenizer
    model, tokenizer = load_model(args.model_path, device)

    if args.prompt:
        # Single prompt mode
        print(f"Prompt: {args.prompt}")
        print("\nResponse:")
        generate_response(
            model,
            tokenizer,
```

```
            args.prompt,
            device,
            max_new_tokens=args.max_new_tokens,
            temperature=args.temperature,
        )
        print("\n" + "=" * 80 + "\n")
    else:
        # Interactive mode
        print("Entering interactive mode. Type 'exit' or 'quit' to end.")
        try:
            while True:
                prompt = input("Prompt: ")
                if prompt.lower() in ["exit", "quit"]:
                    break
                print("\nResponse:")
                generate_response(
                    model,
                    tokenizer,
                    prompt,
                    device,
                    max_new_tokens=args.max_new_tokens,
                    temperature=args.temperature,
                )
                print("\n" + "=" * 80 + "\n")
        except KeyboardInterrupt:
            print("\nExiting...")
        except Exception as e:
            print(f"An error occurred: {e}")


if __name__ == "__main__":
    main()
```

The inference script is written to only use Huggingface libraries only so it's more readily available to run anywhere. It loads the merged model and can either run it interactively or on a single prompt.

The `load_model` function initializes and loads the tokenizer and model. The `format_prompt` function wraps the user input into a structured format suitable for Llama 3's chat-style generation. It defines a system message that informs the model it should act as a helpful assistant knowledgeable in Singapore law. Then it uses the tokenizer's `apply_chat_template` method to generate a properly formatted prompt string.

The `generate_response` function handles the actual inference. It first formats the prompt and tokenizes it into tensors, moves the input to the model's device, and sets up the `TextStreamer` to stream output as it's generated. Then, it calls the model's `generate` method with specified parameters such as maximum new tokens and temperature for sampling. It also specifies end-of-sequence (EOS) tokens to control when generation should stop. The actual response is streamed directly to stdout, so the function returns an empty string since the streamer already handles the output.

Running the inference script

Let's try out our script!

```
$ python infer.py --prompt "Interpret the offence under Section 10 of the Prevention of Corruption Act involving corruptly
Using device: cuda
Loading model from ./llama-3.1-8b-instruct-sg-legislation...
Model loaded successfully.
Prompt: Interpret the offence under Section 10 of the Prevention of Corruption Act involving corruptly procuring or withdra

Response:
Section 10 of the Prevention of Corruption Act in Singapore deals with the offence of corruptly procuring or withdrawing te

**Section 10. Corruptly procuring or withdrawing tenders**

(1) Any person who corruptly, whether for the benefit of himself or of any other person, procures or procures the procureme

(2) Any person who corruptly, whether for the benefit of himself or of any other person, withdraws or attempts to withdraw

(3) Any person who is or has been an officer or servant of the Government or of any government department, who, for the ben

**Definition of "agent"**

In the context of this section, an "agent" includes any person who acts on behalf of another person in dealing with the Gov
```

```
**Penalties prescribed**

The penalties for an offence under Section 10 of the Prevention of Corruption Act are as follows:

* For a first offender, the maximum penalty is a fine of $100,000 or imprisonment for up to 5 years, or both.
* For a second or subsequent offender, the maximum penalty is a fine of $200,000 or imprisonment for up to 7 years, or both

Please note that these penalties may be subject to changes, and it's always best to check the latest laws and regulations f
```

It works!

Some thoughts on fine-tuning

There has been conflicting views whether fine-tuning is difficult or not and for anyone who hasn't actually tried it out, it can be confusing. Now that you have gone through (part of) the journey with me, what do you think?

I personally think it's both easy and difficult (ok that sounds like a cop-out). It's easy because the steps are relatively straightforward, but figuring out the parameters and the configurations can be a bit overwhelming and needs a lot of trial and error. I watched a show called Lego Masters on TV and it was amazing the creativity the brickmasters can achieve with fundamentally just plastic blocks. It just feels a bit like that.

"Yellow" by artist Nathan Sawaya. It's part of The Art of the Brick, an exhibition showcasing intricate sculptures made entirely from LEGO bricks.

At the same time, while the model was fine-tuned properly and the answers seemed correct at first glance, closer inspection shows they're not much better than the base model's. There could be several reasons for this.

Llama-3.1–8B may simply lack the capacity to store and retrieve precise statutory facts, especially since I used LoRA, which only updates a small fraction of the model's weights. This means the model may still fall back on general language patterns from pre-training, leading to confident but legally inaccurate responses.

Another issue is that my dataset only contains correct answers, without examples of incorrect or misleading ones. Without contrastive or rejection examples, the model has no signal to avoid plausible-sounding errors — like fabricating penalties or offences.

The instruction formatting used in the script is generic and doesn't explicitly anchor responses to specific statutes or sections, making it easier for the model to hallucinate rather than quote the law.

Finally, the training process may have been affected by suboptimal hyper-parameters or a lack of section-level evaluation, so it's unclear whether the model actually improved in legal accuracy or just became better at sounding helpful.

Fine-tuning a language model on Singapore legislation can be useful — but it doesn't automatically produce fully accurate legal answers. What it can possibly help with is teaching the model how to write in a legal style, use the right terms, and structure its answers more like a lawyer would. So, the model may sound knowledgeable and give responses that are well formatted. However, it can still get important legal details wrong, especially when it tries to recall specific laws or sections.

For use cases like writing legal drafts, classifying documents, or working offline without access to external tools, fine-tuning can still be valuable. But for tasks that require precise, up-to-date legal facts — like answering statutory questions or quoting penalties — a better solution is combining the fine-tuned model with a retrieval system (aka RAG). This approach pulls in the exact law text when the model answers, reducing mistakes.

In short fine-tuning helps, but it's not enough on its own for legal accuracy. It works best when combined with retrieval and careful engineering.

AI | Llm | Fine Tuning | Llama 3 | Legaltech

Follow

Written by Sau Sheong

3K followers · 249 following

I write, code.

Responses (4)

Erdogan T

What are your thoughts?

Horst Herb
Aug 7

Very nice writeup, easy to follow! However, I wonder whether for this purpose that type of fine tuning is really the best way? What you are essentially doing is mostly lossy compression of a lot of data. Yes, you model will learn to predict along... more

3 Reply

Eric Baker
Aug 3

Thanks for documenting this workflow. Very illustrative and appreciated.

Reply

Zelin's Secrets
Jul 14

LLMs can also be trained on specific legal data — like a country's laws or court cases — so they become even more accurate and useful for that legal system. And this is precisely what I...

Reading this, I couldn't help but think of my first attempt at baking sourdough—how I obsessively tweaked variables like hydration and fermentation times, only to produce loaves that were either bricks or puddles. The article's breakdown of... more

1 reply Reply

See all responses

More from the list: "LLMd"

Curated by Erdogan T

Ramakrushna Mohapatra
Top 10 LLM & RAG Projects for Your AI...
Aug 3

In Coding N... by Civil Lear...
Google's New LLM Runs on Just 0.5 GB RAM — ...
Aug 15

In Level Up C... by Fareed ...
Building the Entire RAG Ecosystem and...
Aug 6

Rohit Kumar Thakur
AI Just Hit a Wall. This Model Smashed Right...
Aug 3

View list

More from Sau Sheong

Sau Sheong
Rust for Go programmers
An introduction to Rust from a Go programmer's perspective

Aug 19 20 1

In Level Up Coding by Sau Sheong

How to create MCP Servers with Go

Writing local and remote MCP Servers using Go

Apr 18 173 1

Sau Sheong

How to Set Up a Local Web Server on macOS 15 Sequoia

At times you want to have a local web server for testing anything, including HTML files, running on your MacOS laptop. You could of course...

Oct 13, 2024 9 6

Sau Sheong

Why design is important in engineering

A brief history of the Unix philosophy

Jun 27 42 1

See all from Sau Sheong

Recommended from Medium

In Python in Plain English by Mayuresh K

Architecture of AI-Driven Systems: What Every Technical Architect Should Know

A Step-by-Step Guide to Integrating External AI Services into Commercial Systems with Fallback Strategies

Aug 6 221 7

In Artificial Intelligence in Plain English by Piyush Agnihotri

Building Agentic RAG with LangGraph: Mastering Adaptive RAG for Production

Build intelligent RAG systems that know when to retrieve documents, search the web, or generate responses directly

Jul 20 1.7K 27

In Towards Deep Learning by Sumit Pandey

DINOv3 Just Changed Computer Vision Forever

I've been waiting for this one. Meta's DINOv3 feels like the moment self-supervised vision finally crosses from great features to a single...

Aug 14 147 4

In AIGuys by Vishal Rajput

Context Engineering Over Prompt Engineering

Prompt Engineering is Dead, it's time for Context Engineering.

Aug 6 309 3

Abhinav

Why Microservices Are Out and Monoliths Are Making a Comeback

For years, we were told microservices were the future.

Jul 29 1.8K 110

Joe Njenga

9 Books Every AI Engineer Should Read (To Go Fully Professional)

Picking an AI engineering book and reading it from start to finish is tough!

Jul 23 404 9

See more recommendations